Individual Prediction Report

Introduction

This report explains how I produced my individual submission for the nine Q0 financial targets. I compared a naive baseline, several non-deep-learning models, and several neural-network architectures using the same five-fold cross-validation procedure. My final submission used a 93% accounting-adjusted XGBoost and 7% accounting-adjusted Random Forest ensemble. It achieved a mean cross-validation sMAPE of 13.9664 with a fold standard deviation of 0.1020.

Data Preparation

I used the 100,000 labeled observations in train.csv to build the models and generated predictions for the 50,000 observations in test_individual.csv. The training data contained nine target variables and 203 identifier and predictor columns. I removed Id from the model inputs because it identifies rows rather than describing a company. After removing it, the modeling data contained 202 predictors: 198 numerical variables and four categorical variables for industry, sector, financial currency, and analyst recommendation.

Before fitting any model, I checked that the training and test predictor columns matched and placed them in the same order. The Id values were unique, and I found no duplicated rows. The initial files did not contain missing values, but I still made the pipeline robust to them. I replaced infinite numerical values with missing values, used median imputation for numerical features when a model required it, and treated a missing categorical value as its own category. I also converted categorical values to a consistent string format and removed extra spaces.

The preprocessing depended on the model. Elastic Net and the neural networks used standardized numerical predictors and one-hot encoded categorical predictors. Random Forest and XGBoost used ordinal encoding for the four categorical predictors and did not require feature scaling. For every cross-validation fold, I estimated imputations, scaling values, and category encodings using only the training portion of that fold. I then applied the fitted transformation to the validation portion. This kept validation information out of the training process.

The nine targets had very different scales and included large outliers. Operating income and EBITDA were also negative for most observations, so a regular logarithm was not appropriate. I used a signed-log transformation that reduced the influence of extreme values while preserving whether a value was positive or negative. After prediction, I reversed the transformation and calculated sMAPE in the original financial units. This kept model training more stable while keeping evaluation aligned with the project metric.

Method Selection

Cross Validation and Evaluation

I compared all models using the same five shuffled folds and a fixed random seed of 937. Each fold trained on 80,000 observations and validated on 20,000 observations. Shuffling was reasonable because each row represented a separate synthetic company observation rather than one point in a single time series. Using identical folds also made the comparisons fair.

For each fold, I calculated sMAPE separately for all nine targets and averaged those values so that every target received equal weight. I report both the mean across folds and the fold standard deviation. The mean measures prediction accuracy, while the standard deviation shows how sensitive a result was to the particular training and validation split. Hyperparameters were compared using these out-of-fold results rather than the hidden test data.

Naive Baseline

The required naive baseline predicted every validation observation using the training-fold mean for that target. It ignored all predictors and achieved a mean cross-validation sMAPE of 135.9331 with a standard deviation of 0.4741. This poor result showed that a single typical value could not represent the large differences among companies and that the historical financial predictors contained substantial useful information.

Non Deep Learning Models

I first used Elastic Net as a regularized linear benchmark. I tested different overall regularization strengths and different balances between its L1 and L2 penalties. The best configuration used an alpha of 0.1 and an L1 ratio of 0.9. It improved the mean sMAPE to 94.6253 with a standard deviation of 0.2182, but it was still far behind the tree-based models. This suggested that a mainly linear relationship was too restrictive for these predictors and targets.

I next compared two Random Forest configurations. The stronger version used 200 trees, a maximum depth of 25, a minimum leaf size of two, and half of the predictors as candidates at each split. It achieved 27.3906 +/- 0.1270. The large improvement over Elastic Net confirmed that nonlinear effects and interactions were important. Random Forest performed especially well for assets, liabilities, revenues, and operating expenses, although operating income and EBITDA remained difficult.

XGBoost was my strongest individual model before post-processing. I tested four configurations that varied tree depth, learning rate, row and feature sampling, and regularization. I trained a separate model for each target and used early stopping within each fold. The selected configuration used a learning rate of 0.03, maximum tree depth of eight, minimum child weight of ten, 80% row sampling, 80% feature sampling, an L1 penalty of 0.1, and an L2 penalty of ten. Its average best iteration was about 2,321 out of the 3,000-round limit. It achieved 23.2273 +/- 0.1455, which was about 15% better than Random Forest.

The target-level results showed where the remaining error came from. XGBoost was accurate for assets, liabilities, revenues, cost of revenues, and operating expenses, but the sMAPE values for operating income and EBITDA were both close to 77. These targets are differences between other financial quantities and are often negative or close to zero. Predicting all nine targets independently did not preserve their accounting relationships after I converted predictions back to the original scale.

Accounting Based Post Processing

I used out-of-fold predictions to test whether some targets should be calculated from stronger component predictions. I calculated gross profit from predicted revenue minus predicted cost of revenues. I then calculated operating income from gross profit minus predicted operating expenses. Finally, I calculated EBITDA from operating income plus depreciation and amortization. These rules used only model predictions and information available in the test data, so they did not introduce leakage.

Q0 depreciation and amortization was not a test predictor, so I compared three ways to estimate it: carrying the Q1 value forward, fitting Ridge regression to its Q1 through Q10 history, and fitting XGBoost with all predictors. Carrying Q1 forward was the simplest and most accurate choice. It achieved a D&A sMAPE of 2.3389 and produced the best EBITDA result. The fitted models added complexity without improving accuracy, which suggested strong quarter-to-quarter persistence.

The accounting adjustments produced the largest improvement in the project. XGBoost improved from 23.2273 to 14.0320, and its fold standard deviation fell to 0.1024. Gross-profit sMAPE improved from 26.5696 to 17.9928, operating-income sMAPE improved from 77.1535 to 40.8649, and EBITDA sMAPE improved from 77.0919 to 39.1992. These gains were much larger than the gains from ordinary hyperparameter changes.

I then blended the accounting-adjusted XGBoost and Random Forest out-of-fold predictions. I tested XGBoost weights from 50% through 100% and chose the weight with the lowest mean cross-validation sMAPE. The best blend used 93% XGBoost and 7% Random Forest. It achieved 13.9664 +/- 0.1020. The small Random

Forest contribution added useful diversity, while the large XGBoost weight confirmed that boosting remained the main source of predictive accuracy.

Deep Learning Models

I also developed neural networks to meet the deep-learning comparison required by the project. The deep-learning pipeline median-imputed numerical variables, added numerical missing-value indicators, standardized the numerical inputs, and one-hot encoded the categorical variables. Each network predicted all nine signed-log targets jointly. I used mean squared error, the AdamW optimizer, He initialization, batch normalization, dropout where appropriate, and fixed random seeds.

My first set of experiments compared six feed-forward designs, including a single wide layer, a four-layer network with different dropout rates, a version trained without early stopping, and a residual network. The four-layer model with 10% dropout provided the best initial balance. However, the training histories showed that patience-based early stopping could stop before later improvements. I therefore used longer training, a learning-rate scheduler, and the checkpoint with the best validation performance.

I then compared a wider network, a reference deep network, and a deeper six-layer network. The wider model used hidden layers of 1,024, 512, 256, and 128 units and achieved 26.5376 +/- 0.3040. It outperformed the reference model at 28.6189 and the deeper model at 28.8613. This indicated that more width was more useful than simply stacking additional layers for this dataset.

I also tested a Hybrid GRU-MLP to see whether the ten historical quarters would benefit from explicit sequence modeling. The GRU summarized the quarterly financial histories, while a separate MLP processed the static and categorical predictors. The combined model achieved 28.7174 +/- 0.5319 and was worse than the wider MLP for every target. The ordinary MLP already received the complete history directly, while the GRU had to compress that history into a smaller representation. The sequence was also fairly short, so the recurrent structure did not provide an advantage.

My final deep-learning experiment compared activation functions and weight decay using the wider architecture. ReLU with weight decay had the lowest mean at 25.5476, but its fold standard deviation was 0.6576. Leaky ReLU with 10% dropout and weight decay of 0.0001 achieved a very similar mean of 25.7305 with a much smaller standard deviation of 0.2183. I selected the Leaky ReLU version as the final deep-learning model because its performance was much more consistent across folds. I trained that configuration on the full training set for 200 epochs.

The neural networks improved substantially over the naive baseline and Elastic Net, but even the selected network was less accurate than unadjusted XGBoost. The result was consistent with the structure of the problem: the data were mainly tabular numerical variables with only four categorical predictors, and tree ensembles captured the nonlinear relationships efficiently without needing to learn a separate internal representation.

Final Individual Model Selection

The full comparison showed a clear progression. The naive baseline scored 135.9331, Elastic Net scored 94.6253, Random Forest scored 27.3906, and unadjusted XGBoost scored 23.2273. The selected deep-learning model scored 25.7305. Accounting-adjusted XGBoost reduced the score to 14.0320, and the 93% XGBoost and 7% Random Forest ensemble produced the best result at 13.9664. Its fold standard deviation of 0.1020 also showed that the result was stable across the five splits.

I therefore used the accounting-adjusted XGBoost and Random Forest ensemble for my individual submission. After making this choice entirely from cross-validation, I retrained both component models on all 100,000 labeled observations. I reversed the signed-log transformation, applied the selected accounting relationships, combined the model predictions with the chosen weights, and restored Id as the first column of the submission. The hidden test data were used only for generating the final predictions.

Individual Conclusion

My main finding was that domain structure mattered more than adding model complexity. XGBoost was the strongest standalone learner, but the largest improvement came from calculating linked financial quantities in a consistent way. A small Random Forest weight then provided a modest additional gain. The neural-network experiments were still useful because they showed that wider feed-forward models were more suitable than deeper or recurrent designs, even though they did not beat the tree ensembles.

The approach also had limitations. Operating income and EBITDA remained the hardest targets because they were often negative or near zero and inherited error from several component predictions. The accounting equations were helpful, but the synthetic data did not always follow them perfectly, and using Q1 depreciation and amortization assumes short-term persistence. The hyperparameter searches were focused rather than exhaustive because full five-fold training was expensive. If I extended the individual work, I would test more efficient boosted-tree searches, model the difficult targets with losses closer to sMAPE, and explore joint models that enforce accounting consistency during training rather than only after prediction.